MODULE 06 | NETWORKING

Networking

Test connections, fetch data, look up names, inspect ports and log in to remote machines.

Mission	Topic	Commands / Concepts
1	My Network: ip, ping and traceroute	ip addr, ip route, ping, traceroute, hostname
2	Talk to the Web: curl and wget	curl, curl -I, -o, -X, -H, -d, wget
3	Name Lookup: dig, nslookup, host	dig, nslookup, host, /etc/hosts, resolv.conf
4	Sockets and Ports: ss	ss, -t -u -l -n -p, lsof, nc
5	Remote Access: ssh, scp and Keys	ssh, scp, rsync, ssh-keygen, ssh-copy-id, ~/.ssh/config
Boss	The Dead Website	Everything in this module

How to use this guide: read each mission, type every example yourself in the Tux Tutor terminal, attempt the practice tasks, then check the answers at the end. Use a throwaway practice folder so mistakes cost nothing.

Module Overview

Key ideas

Term	Meaning
IP address	Numeric address of a machine, e.g. `192.168.1.10` (IPv4) or `2001:db8::1` (IPv6)
Port	Number identifying a service on a machine: `22` SSH, `80` HTTP, `443` HTTPS, `53` DNS
DNS	Translates names (`example.com`) into IP addresses
Gateway	The router that sends your traffic to other networks
TCP / UDP	TCP is reliable and connection-based; UDP is fast and connectionless
localhost	`127.0.0.1`, your own machine

Common ports

Port	Service
22	SSH (secure remote login)
25 / 587	SMTP mail
53	DNS
80 / 443	HTTP / HTTPS
3306	MySQL
5432	PostgreSQL
8080	Common alternative web port

The troubleshooting ladder

When 'the internet is down', test from the bottom up:

```
1. ip addr            do I have an IP?
2. ping gateway        can I reach my router?
3. ping 8.8.8.8       can I reach the internet by IP?
4. ping example.com   does DNS work?
5. curl -I URL        does the web service respond?
6. ss -tuln           is something listening on the port?
```

Safety
Only scan or test machines and networks that you own or have permission to test.

MISSION 1 | +50 XP

My Network: ip, ping and traceroute

Goal: Find your address, test reachability and see the route to a host.

```
$ hostname                    # machine name
$ hostname -I                 # IP addresses
$ ip addr                     # all interfaces (short: ip a)
$ ip -br addr                 # brief one-line view
2: eth0   UP   192.168.1.10/24
$ ip route                    # routing table
default via 192.168.1.1 dev eth0     # 192.168.1.1 is your gateway
$ ip link                     # interface status
```

> **Tip**
> `ifconfig` is the old tool. Prefer `ip`, which is installed on all modern systems.

ping

```
$ ping -c 4 8.8.8.8
PING 8.8.8.8 56 bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=117 time=14.2 ms
...
4 packets transmitted, 4 received, 0% packet loss
rtt min/avg/max = 13.9/14.4/15.1 ms
```

Result	Likely meaning
Replies, low ms	Connection healthy
`Destination Host Unreachable`	No route or host down
`Name or service not known`	DNS problem (try the IP)
Packet loss	Unstable link or congestion
No reply but site works	Host may block ICMP

```
$ ping -c 4 localhost              # test your own network stack
$ ping -c 3 -i 0.5 example.com     # 0.5 s interval
$ traceroute example.com           # every hop on the way
$ tracepath example.com            # similar, needs no root
```

Practice

- Find your IP address and default gateway.
- Ping the gateway, then `8.8.8.8`, then a domain name. Where does it fail, if anywhere?
- Use `traceroute` to see how many hops to a public website.

MISSION 2 | +50 XP

Talk to the Web: curl and wget

Goal: Download files and call web APIs from the terminal.

```
$ curl https://example.com                # print the page body
$ curl -I https://example.com              # headers only
$ curl -s https://example.com | head -5    # -s silent (no progress bar)
$ curl -o page.html https://example.com    # save with chosen name
$ curl -O https://example.com/file.zip     # save with remote name
$ curl -L http://short.link/x              # follow redirects
$ curl -v https://example.com              # verbose (see handshake)
$ curl -w "%{http_code}\n" -o /dev/null -s https://example.com   # just the status
```

HTTP status codes

Code	Meaning
`200`	OK
`301` / `302`	Redirect
`401` / `403`	Not authenticated / forbidden
`404`	Not found
`500` / `502` / `503`	Server error / bad gateway / unavailable

Calling APIs

```
$ curl -s https://api.github.com/users/octocat | head
$ curl -X POST https://httpbin.org/post \
       -H "Content-Type: application/json" \
       -d '{"name":"haani"}'
$ curl -u user:pass https://site/api         # basic auth
$ curl -H "Authorization: Bearer TOKEN" https://site/api
```

wget

```
$ wget https://example.com/file.zip        # download
$ wget -c https://example.com/big.iso       # resume an interrupted download
$ wget -r -np https://example.com/docs/     # recursive download
```

> **Tip**
> `curl` is best for APIs and quick checks; `wget` is best for downloading files and sites.

Practice

- Get only the HTTP status code of a website.
- Download a file with `curl -O`, then with `wget`.
- Send a POST request to `https://httpbin.org/post` with a JSON body.

MISSION 3 | +50 XP

Name Lookup: dig, nslookup, host

Goal: Understand how names become addresses and debug DNS.

```
$ dig example.com
;; ANSWER SECTION:
example.com.   3600   IN   A   93.184.216.34
;; Query time: 22 msec
;; SERVER: 127.0.0.53#53

$ dig +short example.com          # just the answer
$ dig example.com AAAA            # IPv6 record
$ dig example.com MX              # mail servers
$ dig example.com NS              # authoritative name servers
$ dig example.com TXT             # text records (SPF, verification)
$ dig @8.8.8.8 example.com        # ask a specific DNS server
$ dig -x 8.8.8.8                  # reverse lookup (IP to name)
```

Record	Purpose
A	Name to IPv4
AAAA	Name to IPv6
CNAME	Alias to another name
MX	Mail servers
NS	Name servers of the domain
TXT	Free text, email policies
PTR	Reverse lookup

Simpler tools

```
$ host example.com
$ nslookup example.com
```

Local resolution files

```
$ cat /etc/hosts           # manual name -> IP entries, checked first
127.0.0.1   localhost
$ cat /etc/resolv.conf     # which DNS servers are used
$ resolvectl status        # on systemd-resolved systems
```

Tip
If `ping 8.8.8.8` works but `ping example.com` fails, the problem is DNS. Compare `dig example.com` with `dig @8.8.8.8 example.com` to see if your DNS server is the issue.

Practice

- Find the IP address of a website with `dig +short`.
- Find the mail servers of `gmail.com`.

- Add a test line to `/etc/hosts` (with sudo) that maps `myapp.local` to `127.0.0.1` and ping it.

MISSION 4 | +50 XP

Sockets and Ports: ss

Goal: See which services are listening and which connections are open.

```
$ ss -tuln                  # listening TCP+UDP ports, numeric
Netid State  Recv-Q Send-Q Local Address:Port
tcp   LISTEN 0      128    0.0.0.0:22
tcp   LISTEN 0      511    127.0.0.1:8080
$ sudo ss -tulnp              # include the owning process
$ ss -t                     # established TCP connections
$ ss -s                     # summary statistics
$ ss -tn state established '( dport = :443 )'
```

Flag	Meaning
`-t` / `-u`	TCP / UDP
`-l`	Listening sockets only
`-n`	Numeric (no name lookups)
`-p`	Show process (needs sudo for others)
`-a`	All sockets

Reading Local Address

- `0.0.0.0:22` or `*:22` listens on **all interfaces**, reachable from outside.
- `127.0.0.1:8080` listens on **localhost only**, not reachable remotely.
- `[::]:80` is the IPv6 equivalent of all interfaces.

Related tools

```
$ sudo lsof -i :8080            # which process uses port 8080?
$ nc -zv example.com 443        # test if a TCP port is open
Connection to example.com 443 port [tcp/https] succeeded!
$ nc -zv localhost 20-25        # test a range
$ netstat -tuln                 # older equivalent of ss
```

Tip
'Address already in use' errors mean something is already listening. Find the culprit with `ss -tulnp | grep :PORT` and decide whether to stop it.

Practice

- List every listening port on your machine and the process owning each.
- Find what is listening on port 22 (or confirm nothing is).
- Start `python3 -m http.server 8000 &` and verify it appears in `ss`, then fetch it with `curl`.

MISSION 5 | +50 XP

Remote Access: ssh, scp and Keys

Goal: Log in to remote machines securely and transfer files.

```
$ ssh user@192.168.1.20               # log in
$ ssh -p 2222 user@host                 # non-default port
$ ssh user@host "uptime; df -h"         # run commands and return
$ ssh -i ~/.ssh/mykey user@host         # choose a key
$ exit                                  # leave the remote shell
```

First connection asks to trust the server's fingerprint. Answering `yes` stores it in `~/.ssh/known_hosts`. A later **WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED** means the server was rebuilt or something is wrong. Investigate before continuing.

Key-based login (recommended)

```
$ ssh-keygen -t ed25519 -C "haani@laptop"     # creates ~/.ssh/id_ed25519 (+ .pub)
$ ssh-copy-id user@host                       # installs your public key
$ ssh user@host                               # no password needed now
$ chmod 700 ~/.ssh && chmod 600 ~/.ssh/id_ed25519   # required permissions
```

> **Never share**
> Share only the `.pub` file. The private key (without `.pub`) must stay secret.

Copying files

```
$ scp report.txt user@host:/home/user/         # upload
$ scp user@host:/var/log/syslog ./               # download
$ scp -r project/ user@host:~/                   # folder
$ rsync -avz project/ user@host:~/project/       # efficient sync, only changes
```

Shortcuts in ~/.ssh/config

```
Host web
    HostName 192.168.1.20
    User ubuntu
    Port 22
    IdentityFile ~/.ssh/id_ed25519

$ ssh web            # now this is all you type
```

Port forwarding

```
$ ssh -L 8080:localhost:80 user@host    # open remote port 80 at localhost:8080
```

Practice

- Generate an ed25519 key pair and inspect both files.
- Write an `~/.ssh/config` entry for a host and connect by its alias.

- Copy a file to a remote machine (or `localhost` if you run an SSH server) with `scp`.

BOSS BATTLE | +150 XP

Boss: The Dead Website

Goal: combine everything from this module without hints.

Scenario

Users report that `myapp` is unreachable. Walk the troubleshooting ladder and document each result.

Tasks

1. Show your IP address and default gateway.
2. Ping the gateway and `8.8.8.8` with 3 packets each.
3. Verify DNS by resolving `example.com` with `dig +short`, then against `@8.8.8.8`.
4. Request only the HTTP status code of `https://example.com` with curl.
5. Start `python3 -m http.server 8000 &`, then confirm with `ss` that port 8000 is listening.
6. Fetch `http://localhost:8000` with curl and show only the headers.
7. Stop the server and confirm the port closes.
8. Save a one-line summary of each test into `netcheck.txt`.

Sample solution

```
ip -br addr; ip route | grep default
ping -c 3 8.8.8.8
dig +short example.com; dig +short @8.8.8.8 example.com
curl -s -o /dev/null -w "%{http_code}\n" https://example.com
python3 -m http.server 8000 &
ss -tuln | grep :8000
curl -I http://localhost:8000
kill %1; sleep 1; ss -tuln | grep :8000 || echo closed
echo "checks complete" >> netcheck.txt
```

Check Your Understanding

Q1. Which command shows your IP addresses?

Q2. What does `ping -c 4` do?

Q3. Which port does SSH use by default?

Q4. What does `curl -I` show?

Q5. Which `dig` option prints only the answer?

Q6. What does `ss -tuln` list?

Q7. What does `127.0.0.1:8080` in ss mean?

Q8. Which file holds your SSH public key by default (ed25519)?

Q9. Command to copy a file to a server?

Q10. If ping by IP works but by name fails, what is broken?

Answers

#	Answer
1	ip addr (or hostname -I).
2	Sends exactly 4 packets.
3	22.
4	Only the HTTP response headers.
5	+short
6	Listening TCP and UDP ports with numeric addresses.
7	Listening on localhost only.
8	~/.ssh/id_ed25519.pub
9	scp file user@host:/path/
10	DNS.

Quick Reference Cheat Sheet

Command	Purpose	Example
ip addr / route	Interfaces and routes	ip -br addr
ping -c	Reachability	ping -c 4 host
traceroute	Path to host	traceroute host
curl -I / -o / -L	Web requests	curl -I url
wget -c	Download	wget -c url
dig +short	DNS lookup	dig +short MX site
ss -tulnp	Listening ports	sudo ss -tulnp

Command	Purpose	Example
nc -zv	Port test	nc -zv host 443
ssh user@host	Remote login	ssh -p 22 u@h
scp / rsync	Copy files	scp f u@h:~/
ssh-keygen	Make keys	ssh-keygen -t ed25519

Up next: Module 07
Shell Scripting. Automate everything with variables, conditionals, loops and functions.